std::optional<T&>

Steve Downey <sdowney@gmail.com>

Document #: PnnnnRm
Date: 2026-11-10
Project: Programming Language C++
Audience: LEWG

Abstract

We propose to fix a hole intentionally left in `std::optional` —

An optional over a reference such that the post condition on assignment is independent of the engaged state, always producing a rebound reference, and assigning a `U` to a `T` is disallowed by `static_assert` if a `U` can not be bound to a `T&`.

Contents

1 Comparison table **1**

2 Motivation **2**

3 Design **2**

4 Rendering Design In C++ **2**

5 Proposal **2**

6 Wording **2**

7 Impact on the standard **18**

8 Acknowledgments **18**

Changes Since Last Version

- **Changes since R_n-1**
  - One.
- **Changes since R_n-2**
  - Tw

1 Comparison table

1.1 Compare Some Things

This is the convention the ...

```
Cat* cat = find_cat("Fido");
if (cat!=nullptr) { return doit(*cat); }
```

```
std::optional<Cat&> cat = find_cat("Fido");
return cat.and_then(doit);
```

2 Motivation

Other than the standard library's implementation of ...

3 Design

The design is straightforward. The `new_thing<T>` ...

3.1 Relational Operations

The definitions of the relational operators ...

3.2 Trivial construction

Construction of `optional<T&>` should be trivial, because it is straightforward to implement, and `optional<T>` is trivial. Boost is not.

4 Rendering Design In C++

While we value performance, we value safety more.

Errors that can be detected at compile time must be detected at compile time.

New Thing must never construct a temporary, or knowingly take the address of an temporary or part of an temporary.

It is always presumed safe to copy the pointer value from New Thing, since by induction, it is not intrinsically dangling.

5 Proposal

Add a New Thing which solves all the problems.

6 Wording

The proposed changes are relative to the current working draft [**?**].

𝔅.1 Optional objects [optional]

𝔅.𝔅.1 General [optional.general]

1 Subclause 𝔅.1 describes class template `optional` that represents optional objects. An *optional object* is an object that contains the storage for another object and manages the lifetime of this contained object, if any. The contained object may be initialized after the optional object has been initialized, and may be destroyed before the optional object has been destroyed. The initialization state of the contained object is tracked by the optional object.

𝔅.𝔅.2 Header `<optional>` synopsis [optional.syn]

```
// mostly freestanding
#include <compare>              // see compare.syn

namespace std {
  // 𝔅.𝔅.3, class template optional
  template <class T>
  class optional; // partially freestanding

  // 𝔅.𝔅.4, partial specialization of optional for lvalue reference types
  template <class T>
  class optional<T&>; // partially freestanding

  template <class T>
  constexpr bool ranges::enable_view<optional<T>> = true;
  template <class T>
  constexpr auto format_kind<optional<T>> = range_format::disabled;
  template<class T>
  constexpr bool ranges::enable_borrowed_range<optional<T&>> = true;

  template<class T>
    concept is-derived-from-optional = requires(const T& t) {       // exposition only
      []<class U>(const optional<U>&){ }(t);
    };

  // 𝔅.𝔅.5, no-value state indicator
  struct nullopt_t{see below};
  inline constexpr nullopt_t nullopt(unspecified);

  // 𝔅.𝔅.6, class bad_optional_access
  class bad_optional_access;

  // 𝔅.𝔅.7, relational operators
  template<class T, class U>
    constexpr bool operator==(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator!=(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator<(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator>(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator<=(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator>=(const optional<T>&, const optional<U>&);
  template<class T, three_way_comparable_with<T> U>
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const optional<U>&);

  // optional.nullops, comparison with nullopt
  template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept;
  template<class T>
    constexpr strong_ordering operator<=>(const optional<T>&, nullopt_t) noexcept;
```

```
  // optional.comp.with.t, comparison with T
  template<class T, class U> constexpr bool operator==(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator==(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator<(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator<(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator>(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator>(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
  template<class T, class U>
      requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
    constexpr compare_three_way_result_t<T, U>
      operator<=>(const optional<T>&, const U&);

  // ?.?.8, specialized algorithms
  template<class T>
    constexpr void swap(optional<T>&, optional<T>&) noexcept(see below);

  template<class T>
    constexpr optional<decay_t<T>> make_optional(T&&);
  template<class T, class... Args>
    constexpr optional<T> make_optional(Args&&... args);
  template<class T, class U, class... Args>
    constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);

  // ?.?.9, hash support
  template<class T> struct hash;
  template<class T> struct hash<optional<T>>;
}
```

?.?.3 Class template `optional` [optional.optional]

?.?.3.1 General [optional.optional.general]

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type     = T;
    using iterator       = implementation-defined;          // see ?.?.3.6
    using const_iterator = implementation-defined;          // see ?.?.3.6

    // ?.?.3.2, constructors
    constexpr optional() noexcept;
    constexpr optional(nullopt_t) noexcept;
    constexpr optional(const optional&);
    constexpr optional(optional&&) noexcept(see below);
    template<class... Args>
      constexpr explicit optional(in_place_t, Args&&...);
    template<class U, class... Args>
      constexpr explicit optional(in_place_t, initializer_list<U>, Args&&...);
    template<class U = remove_cv_t<T>>
      constexpr explicit(see below) optional(U&&);
    template<class U>
      constexpr explicit(see below) optional(const optional<U>&);
    template<class U>
      constexpr explicit(see below) optional(optional<U>&&);
```

```
    // 𝔤.𝔤.3.3, destructor
    constexpr ~optional();

    // 𝔤.𝔤.3.4, assignment
    constexpr optional& operator=(nullopt_t) noexcept;
    constexpr optional& operator=(const optional&);
    constexpr optional& operator=(optional&&) noexcept(see below);
    template<class U = remove_cv_t<T>> constexpr optional& operator=(U&&);
    template<class U> constexpr optional& operator=(const optional<U>&);
    template<class U> constexpr optional& operator=(optional<U>&&);
    template<class... Args> constexpr T& emplace(Args&&...);
    template<class U, class... Args> constexpr T& emplace(initializer_list<U>, Args&&...);

    // 𝔤.𝔤.3.5, swap
    constexpr void swap(optional&) noexcept(see below);

    // 𝔤.𝔤.3.6, iterator support
    constexpr iterator begin() noexcept;
    constexpr const_iterator begin() const noexcept;
    constexpr iterator end() noexcept;
    constexpr const_iterator end() const noexcept;

    // 𝔤.𝔤.3.7, observers
    constexpr const T* operator->() const noexcept;
    constexpr T* operator->() noexcept;
    constexpr const T& operator*() const & noexcept;
    constexpr T& operator*() & noexcept;
    constexpr T&& operator*() && noexcept;
    constexpr const T&& operator*() const && noexcept;
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr const T& value() const &;                                  // freestanding-deleted
    constexpr T& value() &;                                              // freestanding-deleted
    constexpr T&& value() &&;                                            // freestanding-deleted
    constexpr const T&& value() const &&;                                // freestanding-deleted
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;

    // 𝔤.𝔤.3.8, monadic operations
    template<class F> constexpr auto and_then(F&& f) &;
    template<class F> constexpr auto and_then(F&& f) &&;
    template<class F> constexpr auto and_then(F&& f) const &;
    template<class F> constexpr auto and_then(F&& f) const &&;
    template<class F> constexpr auto transform(F&& f) &;
    template<class F> constexpr auto transform(F&& f) &&;
    template<class F> constexpr auto transform(F&& f) const &;
    template<class F> constexpr auto transform(F&& f) const &&;
    template<class F> constexpr optional or_else(F&& f) &&;
    template<class F> constexpr optional or_else(F&& f) const &;

    // 𝔤.𝔤.3.9, modifiers
    constexpr void reset() noexcept;

  private:
    T* val;          // exposition only
  };

  template<class T>
    optional(T) -> optional<T>;
}
```

[1] Any instance of `optional<T>` at any given time either contains a value or does not contain a value. When an instance of `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the optional

object's *contained value*, is nested within the optional object. When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

2 When an `optional<T>` object contains a value, member `val` points to the contained value.

`T` shall be a type `T` shall be a type other than *cv* `in_place_t` or *cv* `nullopt_t` that meets the *Cpp17Destructible* requirements (`cpp17.destructible`).

3 A type `X` is a *valid contained type* for `optional` if `X` is an lvalue reference type or a complete non-array object type, and `remove_cvref_t<X>` is a type other than `in_place_t` or `nullopt_t`. If a specialization of `optional` is instantiated with a type `T` that is not a valid contained type for `optional`, the program is ill-formed. If `T` is an object type, `T` shall meet the *Cpp17Destructible* requirements (`cpp17.destructible`).

1 ?.?.3.2 Constructors [optional.ctor]

2 The exposition-only variable template *`converts-from-any-cvref`* is used by some constructors for `optional`.

```
template<class T, class W>
constexpr bool converts-from-any-cvref =  // exposition only
  disjunction_v<is_constructible<T, W&>, is_convertible<W&, T>,
                is_constructible<T, W>, is_convertible<W, T>,
                is_constructible<T, const W&>, is_convertible<const W&, T>,
                is_constructible<T, const W>, is_convertible<const W, T>>;
```

```
constexpr optional() noexcept;
constexpr optional(nullopt_t) noexcept;
```

3 *Postconditions*: `*this` does not contain a value.

4 *Remarks*: No contained value is initialized. For every object type `T` these constructors are constexpr constructors.

```
constexpr optional(const optional& rhs);
```

5 *Effects*: If `rhs` contains a value, direct-non-list-initializes the contained value with `*rhs`.

6 *Postconditions*: `rhs.has_value() == this->has_value()`.

7 *Throws*: Any exception thrown by the selected constructor of `T`.

8 *Remarks*: This constructor is defined as deleted unless `is_copy_constructible_v<T>` is `true`. If `is_trivially_copy_constructible_v<T>` is `true`, this constructor is trivial.

```
constexpr optional(optional&& rhs) noexcept(see below);
```

9 *Constraints*: `is_move_constructible_v<T>` is `true`.

10 *Effects*: If `rhs` contains a value, direct-non-list-initializes the contained value with `*std::move(rhs)`~~`std::move(*rhs)`~~. `rhs.has_value()` is unchanged.

11 *Postconditions*: `rhs.has_value() == this->has_value()`.

12 *Throws*: Any exception thrown by the selected constructor of `T`.

13 *Remarks*: The exception specification is equivalent to `is_nothrow_move_constructible_v<T>`. If `is_trivially_move_constructible_v<T>` is `true`, this constructor is trivial.

```
template<class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

14 *Constraints*: `is_constructible_v<T, Args...>` is `true`.

15 *Effects*: Direct-non-list-initializes the contained value with `std::forward<Args>(args)...`.

16 *Postconditions*: `*this` contains a value.

17 *Throws*: Any exception thrown by the selected constructor of `T`.

18 *Remarks*: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class U, class... Args>
  constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

19 *Constraints*: `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

20 *Effects*: Direct-non-list-initializes the contained value with `il, std::forward<Args>(args)...`.

21 *Postconditions*: `*this` contains a value.

22 *Throws*: Any exception thrown by the selected constructor of `T`.

23 *Remarks*: If `T`'s constructor selected for the initialization is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class U = remove_cv_t<T>> constexpr explicit(see below) optional(U&& v);
```

24 *Constraints*:

(24.1) — `is_constructible_v<T, U>` is `true`,

(24.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is `false`,

(24.3) — `is_same_v<remove_cvref_t<U>, optional>` is `false`, and

(24.4) — if `T` is *cv* `bool`, `remove_cvref_t<U>` is not a specialization of `optional`.

25 *Effects*: Direct-non-list-initializes the contained value with `std::forward<U>(v)`.

26 *Postconditions*: `*this` contains a value.

27 *Throws*: Any exception thrown by the selected constructor of `T`.

28 *Remarks*: If `T`'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor. The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

```
template<class U> constexpr explicit(see below) optional(const optional<U>& rhs);
```

29 *Constraints*:

(29.1) — `is_constructible_v<T, const U&>` is `true`, and

(29.2) — if `T` is not *cv* `bool`, *`converts-from-any-cvref`*`<T, optional<U>>` is `false`.

30 *Effects*: If `rhs` contains a value, direct-non-list-initializes the contained value with `*rhs`.

31 *Postconditions*: `rhs.has_value() == this->has_value()`.

32 *Throws*: Any exception thrown by the selected constructor of `T`.

33 *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const U&, T>
```

```
template<class U> constexpr explicit(see below) optional(optional<U>&& rhs);
```

34 *Constraints*:

(34.1) — `is_constructible_v<T, U>` is `true`, and

(34.2) — if `T` is not *cv* `bool`, *`converts-from-any-cvref`*`<T, optional<U>>` is `false`.

35 *Effects*: If `rhs` contains a value, direct-non-list-initializes the contained value with `*std::move(rhs)`~~`std::move(*rhs)`~~. `rhs.has_value()` is unchanged.

36 *Postconditions*: `rhs.has_value() == this->has_value()`.

37 *Throws*: Any exception thrown by the selected constructor of `T`.

38 *Remarks*: The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

𝔉.𝔉.3.3 Destructor [optional.dtor]

𝔉.𝔉.3.4 Assignment [optional.assign]

```
constexpr optional<T>& operator=(nullopt_t) noexcept;
```

1 *Effects*: If `*this` contains a value, calls `val->T::~T()` to destroy the contained value; otherwise no effect.

2 *Postconditions*: `*this` does not contain a value.

3 *Returns*: `*this`.

```
constexpr optional<T>& operator=(const optional& rhs);
```

4 *Effects*: See `optional.assign.copy`.

Table 1 — `optional::operator=(const optional&)` effects [tab:optional.assign.copy]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `*rhs` to the contained value	direct-non-list-initializes the contained value with `*rhs`
`rhs` does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

5 *Postconditions*: `rhs.has_value() == this->has_value()`.

6 *Returns*: `*this`.

7 *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator is defined as deleted unless `is_copy_constructible_v<T>` is `true` and `is_copy_assignable_v<T>` is `true`. If `is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T> && is_trivially_destructible_v<T>` is `true`, this assignment operator is trivial.

```
constexpr optional& operator=(optional&& rhs) noexcept(see below);
```

8 *Constraints*: `is_move_constructible_v<T>` is `true` and `is_move_assignable_v<T>` is `true`.

9 *Effects*: See `optional.assign.move`. The result of the expression `rhs.has_value()` remains unchanged.

Table 2 — `optional::operator=(optional&&)` effects [tab:optional.assign.move]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `*std::move(rhs)` ~~`std::move(*rhs)`~~ to the contained value	direct-non-list-initializes the contained value with `*std::move(rhs)` ~~`std::move(*rhs)`~~
`rhs` does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

10 *Postconditions*: `rhs.has_value() == this->has_value()`.

11 *Returns*: `*this`.

12 *Remarks*: The exception specification is equivalent to:

```
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>
```

13 If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment. If `is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T> && is_trivially_destructible_v<T>` is `true`, this assignment operator is trivial.

```
template<class U = remove_cv_t<T>> constexpr optional& operator=(U&& v);
```

14 *Constraints*:

(14.1) — `is_same_v<remove_cvref_t<U>, optional>` is `false`,

(14.2) — `conjunction_v<is_scalar<T>, is_same<T, decay_t<U>>>` is `false`,

(14.3) — `is_constructible_v<T, U>` is `true`, and

(14.4) — `is_assignable_v<T&, U>` is `true`.

15 *Effects*: If `*this` contains a value, assigns `std::forward<U>(v)` to the contained value; otherwise direct-non-list-initializes the contained value with `std::forward<U>(v)`.

16 *Postconditions*: `*this` contains a value.

17 *Returns*: `*this`.

18 *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(const optional<U>& rhs);
```

19 *Constraints*:

(19.1) — `is_constructible_v<T, const U&>` is `true`,

(19.2) — `is_assignable_v<T&, const U&>` is `true`,

(19.3) — *`converts-from-any-cvref`*`<T, optional<U>>` is `false`,

(19.4) — `is_assignable_v<T&, optional<U>&>` is `false`,

(19.5) — `is_assignable_v<T&, optional<U>&&>` is `false`,

(19.6) — `is_assignable_v<T&, const optional<U>&>` is `false`, and

(19.7) — `is_assignable_v<T&, const optional<U>&&>` is `false`.

20 *Effects*: See `optional.assign.copy.templ`.

Table 3 — `optional::operator=(const optional<U>&)` effects [**tab:optional.assign.copy.templ**]

	`*this` contains a value	**`*this` does not contain a value**
`rhs` contains a value	assigns `*rhs` to the contained value	direct-non-list-initializes the contained value with `*rhs`
`rhs` does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

21 *Postconditions*: `rhs.has_value() == this->has_value()`.

22 *Returns*: `*this`.

23 *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
```

24 *Constraints*:

(24.1) — `is_constructible_v<T, U>` is `true`,

(24.2) — `is_assignable_v<T&, U>` is `true`,

(24.3) — *`converts-from-any-cvref`*`<T, optional<U>>` is `false`,

(24.4) — `is_assignable_v<T&, optional<U>&>` is `false`,

(24.5) — `is_assignable_v<T&, optional<U>&&>` is `false`,

(24.6) — `is_assignable_v<T&, const optional<U>&>` is `false`, and

(24.7) — `is_assignable_v<T&, const optional<U>&&>` is `false`.

25 *Effects*: See `optional.assign.move.templ`. The result of the expression `rhs.has_value()` remains unchanged.

Table 4 — `optional::operator=(optional<U>&&)` effects [tab:optional.assign.move.templ]

	`*this` contains a value	`*this` does not contain a value
`rhs` contains a value	assigns `*std::move(rhs)` ~~`std::move(*rhs)`~~ to the contained value	direct-non-list-initializes the contained value with `*std::move(rhs)` ~~`std::move(*rhs)`~~
`rhs` does not contain a value	destroys the contained value by calling `val->T::~T()`	no effect

26 *Postconditions*: `rhs.has_value() == this->has_value()`.

27 *Returns*: `*this`.

28 *Remarks*: If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment.

```
template<class... Args> constexpr T& emplace(Args&&... args);
```

29 *Mandates*: `is_constructible_v<T, Args...>` is `true`.

30 *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes the contained value with `std::forward<Args>(args)...`.

31 *Postconditions*: `*this` contains a value.

32 *Returns*: A reference to the new contained value.

33 *Throws*: Any exception thrown by the selected constructor of `T`.

34 *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `*val` (if any) has been destroyed.

```
template<class U, class... Args> constexpr T& emplace(initializer_list<U> il, Args&&... args);
```

35 *Constraints*: `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

36 *Effects*: Calls `*this = nullopt`. Then direct-non-list-initializes the contained value with `il, std::forward<Args>(args)...`.

37 *Postconditions*: `*this` contains a value.

38 *Returns*: A reference to the new contained value.

39 *Throws*: Any exception thrown by the selected constructor of `T`.

40 *Remarks*: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `*val` (if any) has been destroyed.

𝔉.𝔉.3.5 Swap [optional.swap]

𝔉.𝔉.3.6 Iterator support [optional.iterators]

𝔉.𝔉.3.7 Observers [optional.observe]

𝔉.𝔉.3.8 Monadic operations [optional.monadic]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

1 Let `U` be `invoke_result_t<F, decltype(*val)>`.

2 *Mandates*: `remove_cvref_t<U>` is a specialization of `optional`.

3 *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), *val);
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

4 Let `U` be `invoke_result_t<F, decltype(std::move(*val))>`.

5 *Mandates*: `remove_cvref_t<U>` is a specialization of `optional`.

6 *Effects*: Equivalent to:

```
if (*this) {
  return invoke(std::forward<F>(f), std::move(*val));
} else {
  return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

7 Let `U` be `remove_cv_t<invoke_result_t<F, decltype(*val)>>`.

8 *Mandates*: ~~`U` is a non-array object type other than `in_place_t` or `nullopt_t`.~~ `U` is a valid contained type for `optional`. The declaration

```
U u(invoke(std::forward<F>(f), *val));
```

is well-formed for some invented variable `u`.

[*Note 1*: There is no requirement that `U` is movable. — *end note*]

9 *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), *val)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

10 Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(*val))>>`.

11 *Mandates*: ~~`U` is a non-array object type other than `in_place_t` or `nullopt_t`.~~ `U` is a valid contained type for `optional`. The declaration

```
U u(invoke(std::forward<F>(f), std::move(*val)));
```

is well-formed for some invented variable `u`.

[*Note 2*: There is no requirement that `U` is movable. — *end note*]

12 *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(*val))`; otherwise, `optional<U>()`.

```
template<class F> constexpr optional or_else(F&& f) const &;
```

13 *Constraints*: `F` models `invocable` and `T` models `copy_constructible`.

14 *Mandates*: `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is `true`.

15 *Effects*: Equivalent to:

```
if (*this) {
  return *this;
} else {
  return std::forward<F>(f)();
}
```

```
template<class F> constexpr optional or_else(F&& f) &&;
```

16 *Constraints*: `F` models `invocable` and `T` models `move_constructible`.

17 *Mandates*: `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is `true`.

18 *Effects*: Equivalent to:

```
if (*this) {
  return std::move(*this);
} else {
  return std::forward<F>(f)();
}
```

𝔉.𝔉.3.9 Modifiers [optional.mod]

𝔉.𝔉.4 Partial specialization of optional for reference types [optional.optional.ref]

𝔉.𝔉.4.1 General [optional.optional.ref.general]

```
namespace std {
  template<class T>
  class optional<T&> {
    public:
      using value_type     = T;
      using iterator       = implementation-defined; // see 𝔉.𝔉.4.5

    public:
      // 𝔉.𝔉.4.2, constructors
      constexpr optional() noexcept = default;
      constexpr optional(nullopt_t) noexcept : optional() {}
      constexpr optional(const optional& rhs) noexcept = default;

      template <class Arg>
      constexpr explicit optional(in_place_t, Arg&& arg);
      template <class U>
      constexpr explicit(see below) optional(U&& u) noexcept(see below);
      template <class U>
      constexpr explicit(see below) optional(optional<U>& rhs) noexcept(see below);
      template <class U>
      constexpr explicit(see below) optional(const optional<U>& rhs) noexcept(see below);
      template <class U>
      constexpr explicit(see below) optional(optional<U>&& rhs) noexcept(see below);
      template <class U>
      constexpr explicit(see below) optional(const optional<U>&& rhs) noexcept(see below);

      constexpr ~optional() = default;

      // 𝔉.𝔉.4.3, assignment
      constexpr optional& operator=(nullopt_t) noexcept;
      constexpr optional& operator=(const optional& rhs) noexcept = default;

      template <class U> constexpr T& emplace(U&& u) noexcept(see below);

      // 𝔉.𝔉.4.4, swap
      constexpr void swap(optional& rhs) noexcept;

      // 𝔉.𝔉.3.6, iterator support
      constexpr iterator begin() const noexcept;
      constexpr iterator end() const noexcept;

      // 𝔉.𝔉.4.6, observers
      constexpr T*       operator->() const noexcept;
      constexpr T&       operator*() const noexcept;
      constexpr explicit operator bool() const noexcept;
      constexpr bool     has_value() const noexcept;
      constexpr T&       value() const;                                  // freestanding-deleted
      template<class U = remove_cv_t<T>> constexpr remove_cv_t<T> value_or(U&& u) const;

      // 𝔉.𝔉.4.7, monadic operations
      template <class F> constexpr auto and_then(F&& f) const;
      template <class F> constexpr optional<invoke_result_t<F, T&>> transform(F&& f) const;
```

```
    template <class F> constexpr optional or_else(F&& f) const;

    // 𝔅.𝔅.4.8, modifiers
    constexpr void reset() noexcept;

  private:
    T* val = nullptr; // exposition only

    // 𝔅.𝔅.4.9, exposition only helper functions
    template <class U>
    constexpr void convert-ref-init-val(U&& u); // exposition only
  };
}
```

1 An object of `optional<T&>` *contains a value* if and only if `val != nullptr` is `true`. When an `optional<T&>` contains a value, the *contained value* is a reference to `*val`.

𝔅.𝔅.4.2 Constructors [optional.ref.ctor]

```
template <class Arg>
constexpr explicit optional(in_place_t, Arg&& arg);
```

1 *Constraints*:

(1.1) — `is_constructible_v<T&, Arg>` is `true`, and

(1.2) — `reference_constructs_from_temporary_v<T&, Arg>` is `false`.

2 *Effects*: Equivalent to: *convert-ref-init-val*`(std::forward<Arg>(arg))`.

3 *Postconditions*: `*this` contains a value.

```
template <class U>
constexpr explicit(!is_convertible_v<U, T&>)
optional(U&& u) noexcept(is_nothrow_constructible_v<T&, U>);
```

4 *Constraints*:

(4.1) — `is_same_v<remove_cvref_t<U>, optional>` is `false`,

(4.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is `false`, and

(4.3) — `is_constructible_v<T&, U>` is `true`.

5 *Effects*: Equivalent to: *convert-ref-init-val*`(std::forward<U>(u))`.

6 *Postconditions*: `*this` contains a value.

7 *Remarks*: This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, U>
```

is `true`.

```
template <class U>
constexpr explicit(!is_convertible_v<U&, T&>)
optional(optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, U&>);
```

8 *Constraints*:

(8.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is `false`,

(8.2) — `is_same_v<T&, U>` is `false`, and

(8.3) — `is_constructible_v<T&, U&>` is `true`.

9 *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*rhs);
```

10 *Remarks*: This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, U&>
```

is `true`.

```
template <class U>
constexpr explicit(!is_convertible_v<const U&, T&>)
optional(const optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, const U&>);
```

11 *Constraints*:

(11.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is `false`,

(11.2) — `is_same_v<T&, U>` is `false`, and

(11.3) — `is_constructible_v<T&, const U&>` is `true`.

12 *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*rhs);
```

13 *Remarks*: This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, const U&>
```

is `true`.

```
template <class U>
constexpr explicit(!is_convertible_v<U, T&>)
optional(optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, U>);
```

14 *Constraints*:

(14.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is `false`,

(14.2) — `is_same_v<T&, U>` is `false`, and

(14.3) — `is_constructible_v<T&, U>` is `true`.

15 *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*std::move(rhs));
```

16 *Remarks*: This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, U>
```

is `true`.

```
template <class U>
constexpr explicit(!is_convertible_v<const U, T&>)
optional(const optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, const U>);
```

17 *Constraints*:

(17.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is `false`, and

(17.2) — `is_same_v<T&, U>` is `false`.

(17.3) — `is_constructible_v<T&, const U>` is `true`,

18 *Effects*: Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(*std::move(rhs));
```

19 *Remarks*: This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, const U>
```

is `true`.

𝔉.𝔉.4.3 Assignment [optional.ref.assign]

```
constexpr optional& operator=(nullopt_t) noexcept;
```

1 *Effects*: Assigns `nullptr` to *val*.

2 *Postconditions*: `*this` does not contain a value.

3 *Returns*: `*this`.

```
template <class U>
constexpr T& emplace(U&& u) noexcept(is_nothrow_constructible_v<T&, U>);
```

4 *Constraints*:

(4.1) — `is_constructible_v<T&, U>` is `true`, and

(4.2) — `reference_constructs_from_temporary_v<T&, U>` is `false`.

5 *Effects*: Equivalent to: *`convert-ref-init-val`*`(std::forward<U>(u))`.

𝔓.𝔓.4.4 Swap [optional.ref.swap]

```
constexpr void swap(optional& rhs) noexcept;
```

1 *Effects*: Equivalent to: `swap(`*`val`*`, rhs.`*`val`*`)`.

𝔓.𝔓.4.5 Iterator support [optional.ref.iterators]

```
using iterator = implementation-defined;
```

1 This type models `contiguous_iterator`, meets the *Cpp17RandomAccessIterator* requirements, and meets the requirements for constexpr iterators, with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator`.

2 All requirements on container iterators apply to `optional::iterator`.

```
constexpr iterator begin() const noexcept;
```

3 *Returns*: If `has_value()` is `true`, an iterator referring to *`*val`*. Otherwise, a past-the-end iterator value.

```
constexpr iterator end() const noexcept;
```

4 *Returns*: `begin() + has_value()`.

𝔓.𝔓.4.6 Observers [optional.ref.observe]

```
constexpr T* operator->() const noexcept;
```

1 *Hardened preconditions*: `has_value()` is `true`.

2 *Returns*: *`val`*.

```
constexpr T& operator*() const noexcept;
```

3 *Hardened preconditions*: `has_value()` is `true`.

4 *Returns*: *`*val`*.

```
constexpr explicit operator bool() const noexcept;
```

5 *Returns*: *`val`* `!= nullptr`.

```
constexpr bool has_value() const noexcept;
```

6 *Returns*: *`val`* `!= nullptr`.

```
constexpr T& value() const;
```

7 *Effects*: Equivalent to:

```
return has_value() ? *val : throw bad_optional_access();
```

```
template<class U = remove_cv_t<T>> constexpr remove_cv_t<T> value_or(U&& u) const;
```

8 Let `X` be `remove_cv_t<T>`.

9 *Mandates*: `is_constructible_v<X, T&> && is_convertible_v<U, X>` is `true`.

10 *Effects*: Equivalent to:

```
return has_value() ? *val : static_cast<X>(std::forward<U>(u));
```

𝔉.𝔉.4.7 Monadic operations [optional.ref.monadic]

```
template <class F>
constexpr auto and_then(F&& f) const;
```

1 Let `U` be `invoke_result_t<F, T&>`.

2 *Mandates*: `remove_cvref_t<U>` is a specialization of `optional`.

3 *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), *val);
else
    return remove_cvref_t<U>();
```

```
template <class F>
constexpr optional<remove_cv_t<invoke_result_t<F, T&>>> transform(F&& f) const;
```

4 Let `U` be `remove_cv_t<invoke_result_t<F, T&>>`.

5 *Mandates*: The declaration

```
U u(invoke(std::forward<F>(f), *val));
```

is well-formed for some invented variable `u`.

[*Note 1*: There is no requirement that `U` is movable. — *end note*]

6 *Returns*: If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), *val)`; otherwise, `optional<U>()`.

```
template <class F>
constexpr optional or_else(F&& f) const;
```

7 *Constraints*: `F` models `invocable`.

8 *Mandates*: `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is `true`.

9 *Effects*: Equivalent to:

```
if (has_value())
    return *val;
else
    return std::forward<F>(f)();
```

𝔉.𝔉.4.8 Modifiers [optional.ref.mod]

```
constexpr void reset() noexcept;
```

1 *Effects*: Assigns `nullptr` to *val*.

2 *Postconditions*: `*this` does not contain a value.

𝔉.𝔉.4.9 Exposition only helper functions [optional.ref.expos]

```
template <class U>
constexpr void convert-ref-init-val(U&& u);  // exposition only
```

1 *Effects*: Creates a variable `r` as if by `T& r(std::forward<U>(u));` and then initializes *val* with `addressof(r)`.

𝔉.𝔉.5 No-value state indicator [optional.nullopt]

𝔉.𝔉.6 Class `bad_optional_access` [optional.bad.access]

𝔉.𝔉.7 Relational operators [optional.relops]

𝔉.𝔉.8 Specialized algorithms [optional.specalg]

```
template<class T>
  constexpr void swap(optional<T>& x, optional<T>& y) noexcept(noexcept(x.swap(y)));
```

1 *Constraints*: `is_move_constructible_v<T>` is `true` and `is_swappable_v<T>` is `true`.

Constraints: `is_reference_v<T> || (is_move_constructible_v<T> && is_swappable_v<T>)` is `true`.

2 *Effects*: Calls `x.swap(y)`.

```
template<class T> constexpr optional<decay_t<T>> make_optional(T&& v);
```

3 *Constraints*: The call to `make_optional` does not use an explicit *template-argument-list* that begins with a type *template-argument*.

4 *Returns*: `optional<decay_t<T>>(std::forward<T>(v))`.

```
template<class T, class...Args>
  constexpr optional<T> make_optional(Args&&... args);
```

5 *Effects*: Equivalent to: `return optional<T>(in_place, std::forward<Args>(args)...);`

```
template<class T, class U, class... Args>
  constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);
```

6 *Effects*: Equivalent to: `return optional<T>(in_place, il, std::forward<Args>(args)...);`

𝔉.𝔉.9 Hash support [optional.hash]

𝔉.𝔉.10 Feature-test macro [version.syn]

Change the values of the following macro definition to [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_freestanding_optional 20XXXXL // also in <optional>
#define __cpp_lib_optional              20XXXXL // also in <optional>
```

7 Impact on the standard

A pure library extension, affecting no other parts of the library or language.

8 Acknowledgments

Many thanks to all of the reviewers and authors

Document history

- **Changes since R4**
  - feature test macro
- **Changes since R3**
  - discussion
- **Changes since R1**
  - Design points called out
- **Changes since R0**
  - Wording Updates

Implementation

```
// ----------------------
// BASE AND DETAILS ELIDED
// ----------------------

/****************/
/* optional<T&> */
/****************/

template <class T>
class optional<T&> {
  public:
    using value_type = T;
    using iterator =
        std::contiguous_iterator<T,
                                 optional>; // see [optionalref.iterators]
  public:
    // \ref{optionalref.ctor}, constructors

    constexpr optional() noexcept = default;
    constexpr optional(nullopt_t) noexcept : optional() {}
    constexpr optional(const optional& rhs) noexcept = default;

    template <class Arg>
        requires(std::is_constructible_v<T&, Arg> &&
                 !std::reference_constructs_from_temporary_v<T&, Arg>)
    constexpr explicit optional(in_place_t, Arg&& arg);

    template <class U>
        requires(std::is_constructible_v<T&, U> &&
                 !(std::is_same_v<std::remove_cvref_t<U>, in_place_t>) &&
                 !(std::is_same_v<std::remove_cvref_t<U>, optional>) &&
                 !std::reference_constructs_from_temporary_v<T&, U>)
    constexpr explicit(!std::is_convertible_v<U, T&>)
        optional(U&& u) noexcept(std::is_nothrow_constructible_v<T&, U>) {
        convert_ref_init_val(u);
    }

    template <class U>
        requires(std::is_constructible_v<T&, U> &&
                 !(std::is_same_v<std::remove_cvref_t<U>, in_place_t>) &&
                 !(std::is_same_v<std::remove_cvref_t<U>, optional>) &&
                 std::reference_constructs_from_temporary_v<T&, U>)
    constexpr optional(U&& u) = delete;

    // The full set of 4 overloads on optional<U> by value category, doubled to
    // 8 by deleting if reference_constructs_from_temporary_v is true. This
    // allows correct constraints by propagating the value category from the
    // optional to the value within the rhs.
    template <class U>
        requires(std::is_constructible_v<T&, U&> &&
                 !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
                 !std::is_same_v<T&, U> &&
                 !std::reference_constructs_from_temporary_v<T&, U&>)
    constexpr explicit(!std::is_convertible_v<U&, T&>) optional(
        optional<U>& rhs) noexcept(std::is_nothrow_constructible_v<T&, U&>);
```

```
template <class U>
    requires(std::is_constructible_v<T&, const U&> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, const U&>)
constexpr explicit(!std::is_convertible_v<const U&, T&>)
    optional(const optional<U>& rhs) noexcept(
        std::is_nothrow_constructible_v<T&, const U&>);

template <class U>
    requires(std::is_constructible_v<T&, U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, U>)
constexpr explicit(!std::is_convertible_v<U, T&>)
    optional(optional<U>&& rhs) noexcept(
        noexcept(std::is_nothrow_constructible_v<T&, U>));

template <class U>
    requires(std::is_constructible_v<T&, const U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, const U>)
constexpr explicit(!std::is_convertible_v<const U, T&>)
    optional(const optional<U>&& rhs) noexcept(
        noexcept(std::is_nothrow_constructible_v<T&, const U>));

template <class U>
    requires(std::is_constructible_v<T&, U&> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             std::reference_constructs_from_temporary_v<T&, U&>)
constexpr optional(optional<U>& rhs) = delete;

template <class U>
    requires(std::is_constructible_v<T&, const U&> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             std::reference_constructs_from_temporary_v<T&, const U&>)
constexpr optional(const optional<U>& rhs) = delete;

template <class U>
    requires(std::is_constructible_v<T&, U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             std::reference_constructs_from_temporary_v<T&, U>)
constexpr optional(optional<U>&& rhs) = delete;

template <class U>
    requires(std::is_constructible_v<T&, const U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             std::reference_constructs_from_temporary_v<T&, const U>)
constexpr optional(const optional<U>&& rhs) = delete;

// \ref{optionalref.dtor}, destructor
constexpr ~optional() = default;

// \ref{optionalref.assign}, assignment
```

```
    constexpr optional& operator=(nullopt_t) noexcept;

    constexpr optional& operator=(const optional& rhs) noexcept = default;

    template <class U>
        requires(std::is_constructible_v<T&, U> &&
                 !std::reference_constructs_from_temporary_v<T&, U>)
    constexpr T&
    emplace(U&& u) noexcept(std::is_nothrow_constructible_v<T&, U>);

    // \ref{optionalref.swap}, swap
    constexpr void swap(optional& rhs) noexcept;

    // \ref{optional.iterators}, iterator support
    constexpr iterator begin() const noexcept;
    constexpr iterator end() const noexcept;

    // \ref{optionalref.observe}, observers
    constexpr T*       operator->() const noexcept;
    constexpr T&       operator*() const noexcept;
    constexpr explicit operator bool() const noexcept;
    constexpr bool     has_value() const noexcept;
    constexpr T&       value() const;
    template <class U = std::remove_cv_t<T>>
    constexpr std::remove_cv_t<T> value_or(U&& u) const;

    // \ref{optionalref.monadic}, monadic operations
    template <class F>
    constexpr auto and_then(F&& f) const;
    template <class F>
    constexpr optional<std::invoke_result_t<F, T&>> transform(F&& f) const;
    template <class F>
    constexpr optional or_else(F&& f) const;

    // \ref{optional.mod}, modifiers
    constexpr void reset() noexcept;

  private:
    T* value_ = nullptr; // exposition only

    // \ref{optionalref.expos}, exposition only helper functions
    template <class U>
    constexpr void convert_ref_init_val(U&& u) {
        // Creates a variable, \tcode{r},
        // as if by \tcode{T\& r(std::forward<U>(u));}
        // and then initializes \exposid{val} with \tcode{addressof(r)}
        T& r(std::forward<U>(u));
        value_ = std::addressof(r);
    }
};

//  \rSec3[optionalref.ctor]{Constructors}
template <class T>
template <class Arg>
    requires(std::is_constructible_v<T&, Arg> &&
             !std::reference_constructs_from_temporary_v<T&, Arg>)
constexpr optional<T&>::optional(in_place_t, Arg&& arg) {
    convert_ref_init_val(std::forward<Arg>(arg));
}
```

```
// Clang is unhappy with the out-of-line definition
//
// template <class T>
// template <class U>
//     requires(std::is_constructible_v<T&, U> &&
//     !(is_same_v<remove_cvref_t<U>, in_place_t>) &&
//               !(is_same_v<remove_cvref_t<U>, optional<T&>>) &&
//               !std::reference_constructs_from_temporary_v<T&, U>)
// constexpr optional<T&>::optional(U&& u)
// noexcept(is_nothrow_constructible_v<T&, U>)
//     : value_(std::addressof(static_cast<T&>(std::forward<U>(u)))) {}

template <class T>
template <class U>
    requires(std::is_constructible_v<T&, U&> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, U&>)
constexpr optional<T&>::optional(optional<U>& rhs) noexcept(
    std::is_nothrow_constructible_v<T&, U&>) {
    if (rhs.has_value()) {
        convert_ref_init_val(*rhs);
    }
}

template <class T>
template <class U>
    requires(std::is_constructible_v<T&, const U&> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, const U&>)
constexpr optional<T&>::optional(const optional<U>& rhs) noexcept(
    std::is_nothrow_constructible_v<T&, const U&>) {
    if (rhs.has_value()) {
        convert_ref_init_val(*rhs);
    }
}

template <class T>
template <class U>
    requires(std::is_constructible_v<T&, U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, U>)
constexpr optional<T&>::optional(optional<U>&& rhs) noexcept(
    noexcept(std::is_nothrow_constructible_v<T&, U>)) {
    if (rhs.has_value()) {
        convert_ref_init_val(*std::move(rhs));
    }
}

template <class T>
template <class U>
    requires(std::is_constructible_v<T&, const U> &&
             !std::is_same_v<std::remove_cv_t<T>, optional<U>> &&
             !std::is_same_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, const U>)
constexpr optional<T&>::optional(const optional<U>&& rhs) noexcept(
```

```
    noexcept(std::is_nothrow_constructible_v<T&, const U>)) {
    if (rhs.has_value()) {
        convert_ref_init_val(*std::move(rhs));
    }
}

// \rSec3[optionalref.assign]{Assignment}
template <class T>
constexpr optional<T&>& optional<T&>::operator=(nullopt_t) noexcept {
    value_ = nullptr;
    return *this;
}

template <class T>
template <class U>
    requires(std::is_constructible_v<T&, U> &&
             !std::reference_constructs_from_temporary_v<T&, U>)
constexpr T&
optional<T&>::emplace(U&& u) noexcept(std::is_nothrow_constructible_v<T&, U>) {
    convert_ref_init_val(std::forward<U>(u));
    return *value_;
}

//   \rSec3[optionalref.swap]{Swap}

template <class T>
constexpr void optional<T&>::swap(optional<T&>& rhs) noexcept {
    std::swap(value_, rhs.value_);
}

// \rSec3[optionalref.iterators]{Iterator Support}

template <class T>
constexpr optional<T&>::iterator optional<T&>::begin() const noexcept {
    return iterator(has_value() ? value_ : nullptr);
};

template <class T>
constexpr optional<T&>::iterator optional<T&>::end() const noexcept {
    return begin() + has_value();
}

// \rSec3[optionalref.observe]{Observers}
template <class T>
constexpr T* optional<T&>::operator->() const noexcept {
    return value_;
}

template <class T>
constexpr T& optional<T&>::operator*() const noexcept {
    return *value_;
}

template <class T>
constexpr optional<T&>::operator bool() const noexcept {
    return value_ != nullptr;
}
template <class T>
constexpr bool optional<T&>::has_value() const noexcept {
```

```
    return value_ != nullptr;
}

template <class T>
constexpr T& optional<T&>::value() const {
    return has_value() ? *value_ : throw bad_optional_access();
}

template <class T>
template <class U>
constexpr std::remove_cv_t<T> optional<T&>::value_or(U&& u) const {
    static_assert(std::is_constructible_v<std::remove_cv_t<T>, T&>,
                  "T must be constructible from a T&");
    static_assert(std::is_convertible_v<U, std::remove_cv_t<T>>,
                  "Must be able to convert u to T");
    return has_value() ? *value_
                       : static_cast<std::remove_cv_t<T>>(std::forward<U>(u));
}

//   \rSec3[optionalref.monadic]{Monadic operations}
template <class T>
template <class F>
constexpr auto optional<T&>::and_then(F&& f) const {
    using U = std::invoke_result_t<F, T&>;
    static_assert(detail::is_optional<U>, "F must return an optional");
    if (has_value()) {
        return std::invoke(std::forward<F>(f), *value_);
    } else {
        return std::remove_cvref_t<U>();
    }
}

template <class T>
template <class F>
constexpr optional<std::invoke_result_t<F, T&>>
optional<T&>::transform(F&& f) const {
    using U = std::invoke_result_t<F, T&>;
    static_assert(!std::is_same_v<std::remove_cvref_t<U>, in_place_t>,
                  "Result must not be in_place_t");
    static_assert(!std::is_same_v<std::remove_cvref_t<U>, nullopt_t>,
                  "Result must not be nullopt_t");
    static_assert((std::is_object_v<U> && !std::is_array_v<U>) ||
                      std::is_lvalue_reference_v<U>,
                  "Result must be an non-array object or an lvalue reference");
    if (has_value()) {
        return optional<U>{std::invoke(std::forward<F>(f), *value_)};
    } else {
        return optional<U>{};
    }
}

template <class T>
template <class F>
constexpr optional<T&> optional<T&>::or_else(F&& f) const {
    using U = std::invoke_result_t<F>;
    static_assert(std::is_same_v<std::remove_cvref_t<U>, optional>,
                  "Result must be an optional");
    if (has_value()) {
        return *value_;
```

```
    } else {
        return std::forward<F>(f)();
    }
}

// \rSec3[optional.mod]{modifiers}
template <class T>
constexpr void optional<T&>::reset() noexcept {
    value_ = nullptr;
}
} // namespace beman::optional

namespace std {
template <typename T>
    requires requires(T a) {
        {
            std::hash<remove_const_t<T>>{}(a)
        } -> std::convertible_to<std::size_t>;
    }
struct hash<beman::optional::optional<T>> {
    static_assert(!is_reference_v<T>,
                  "hash is not enabled for reference types");
    size_t operator()(const beman::optional::optional<T>& o) const
        noexcept(noexcept(hash<remove_const_t<T>>{}(*o))) {
        if (o) {
            return std::hash<std::remove_const_t<T>>{}(*o);
        } else {
            return 0;
        }
    }
};
```